

ATIVIDADE AVALIATIVA 2 OPENGL - Sala de cinema

Maria Clara Souza Muharem **RA:** 266864

Ana Vitória Cardoso Queiroz **RA:** 1727064

```
C/C++

// Ana Vitória Cardoso Queiroz - 172706
// Maria Clara Muharem - 266864

#include <GL/glut.h>

//controle camera
float anguloY = 0.0;
float anguloX = 0.0;
float distancia = 15.0;

int ultimoX, ultimoY;
int botaoPressionado = 0;

//objetos da cena
void desenharChao() {
    glColor3f(0.2, 0.2, 0.2);
    glPushMatrix();
    glScalef(12, 0.1, 12);
    glutSolidCube(1);
    glPopMatrix();
}

void desenharTeto() {
    glColor3f(0.15, 0.15, 0.15);
    glPushMatrix();
    glTranslatef(0, 6, 0);
    glScalef(12, 0.1, 12);
    glutSolidCube(1);
    glPopMatrix();
}

void desenharParedes() {
    glColor3f(0.1, 0.1, 0.1);

    glPushMatrix();
    glTranslatef(0, 3, -6);
    glScalef(12, 6, 0.2);
    glutSolidCube(1);
    glPopMatrix();

    glPushMatrix();
    glTranslatef(-6, 3, 0);
    glScalef(0.2, 6, 12);
    glutSolidCube(1);
    glPopMatrix();
}
```

```
        glPushMatrix();
        glTranslatef(6, 3, 0);
        glScalef(0.2, 6, 12);
        glutSolidCube(1);
        glPopMatrix();
    }

    void desenharTela() {
        glColor3f(0.9, 0.9, 0.9);
        glPushMatrix();
        glTranslatef(0, 4, -5);
        glScalef(8, 4, 0.2);
        glutSolidCube(1);
        glPopMatrix();
    }

    void desenharCadeira() {
        glColor3f(0.6, 0.0, 0.0);
        glPushMatrix();
        glScalef(0.8, 0.3, 0.8);
        glutSolidCube(1);
        glPopMatrix();

        glPushMatrix();
        glTranslatef(0, 0.5, 0.3);
        glScalef(0.8, 1.0, 0.2);
        glutSolidCube(1);
        glPopMatrix();
    }

    void desenharEscadas() {
        glColor3f(0.3, 0.3, 0.3);
        int linha = 0;

        for(int z = -2; z <= 4; z += 2) {
            float altura = linha * 0.7;

            glPushMatrix();
            glTranslatef(0, altura + 0.1, z);
            glScalef(12, 0.1, 1.8);
            glutSolidCube(1);
            glPopMatrix();
            linha++;
        }
    }

    void desenharParedeEscada() {
        glColor3f(0.08, 0.08, 0.08);
        glPushMatrix();
        glTranslatef(0, 1.15, 5.0);
        glScalef(12, 2.0, 0.2);
        glutSolidCube(1);
        glPopMatrix();
    }
```

```
}

void desenharLuzesEscada() {
    glColor3f(1.0, 0.9, 0.0);

    int linha = 0;
    for(float z = -2; z <= 4; z += 2) {
        float altura = linha * 0.7;
        float posicoesX[] = {-5.5, -1.0, 1.0, 5.5};

        for(int i = 0; i < 4; i++) {
            glPushMatrix();
            glTranslatef(posicoesX[i], altura + 0.17, z + 0.9);
            glScalef(0.4, 0.02, 0.1);
            glutSolidCube(1);
            glPopMatrix();
        }
        linha++;
    }
}

void desenharCadeiras() {
    int linha = 0;

    for(int z = -2; z <= 4; z += 2) {
        float altura = linha * 0.64;
        for(int x = -4; x <= 4; x += 2) {
            glPushMatrix();
            glTranslatef(x, altura + 0.6, z);
            desenharCadeira();
            glPopMatrix();
        }
        linha++;
    }
}

void desenharLuzes() {
    glColor3f(1.0, 1.0, 0.0);

    for(int i = -4; i <= 4; i += 2) {
        glPushMatrix();
        glTranslatef(-5.5, 3, i);
        glutSolidSphere(0.24, 20, 20);
        glPopMatrix();

        glPushMatrix();
        glTranslatef(5.5, 3, i);
        glutSolidSphere(0.3, 20, 20);
        glPopMatrix();
    }
}

void desenharCaixasSom() {
    glColor3f(0.05, 0.05, 0.05);
```

```
    glPushMatrix();
    glTranslatef(-4.5, 5, -4);
    glScalef(0.6, 1.2, 0.6);
    glutSolidCube(1);
    glPopMatrix();

    glPushMatrix();
    glTranslatef(4.5, 5, -4);
    glScalef(0.6, 1.2, 0.6);
    glutSolidCube(1);
    glPopMatrix();

    glColor3f(0.2, 0.2, 0.2);

    glPushMatrix();
    glTranslatef(-4.5, 5, -3.5);
    glutSolidSphere(0.2, 20, 20);
    glPopMatrix();

    glPushMatrix();
    glTranslatef(4.5, 5, -3.5);
    glutSolidSphere(0.2, 20, 20);
    glPopMatrix();
}

void desenharaPorta() {
    glColor3f(0.05, 0.05, 0.05);
    glPushMatrix();
    glTranslatef(5.9, 1.5, -4.0);
    glScalef(0.1, 3.0, 3.5);
    glutSolidCube(1);
    glPopMatrix();

    glColor3f(0.2, 0.1, 0.05);
    glPushMatrix();
    glTranslatef(5.85, 1.5, -4.0);
    glScalef(0.05, 2.9, 3.2);
    glutSolidCube(1);
    glPopMatrix();

    glColor3f(0.8, 0.7, 0.2);
    glPushMatrix();
    glTranslatef(5.75, 1.5, -3.0);
    glutSolidSphere(0.08, 10, 10);
    glPopMatrix();
}

//display
void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glLoadIdentity();
    gluLookAt(0, 5, distancia,
```

```
        0, 0, 0,
        0, 1, 0);

glRotatef(anguloX, 1, 0, 0);
glRotatef(anguloY, 0, 1, 0);

desenharChao();
desenharTeto();
desenharParedes();
desenharPorta();
desenharTela();
desenharEscadas();
desenharLuzesEscada();
desenharCadeiras();
desenharParedeEscada();
desenharLuzes();
desenharCaixasSom();

glutSwapBuffers();
glutPostRedisplay();
}

// mouse clique
void mouse(int button, int state, int x, int y) {
    if (button == GLUT_LEFT_BUTTON) {
        if (state == GLUT_DOWN) {
            botaoPressionado = 1;
            ultimoX = x;
            ultimoY = y;
        } else {
            botaoPressionado = 0;
        }
    }

    if (button == 3) distancia -= 1.0;
    if (button == 4) distancia += 1.0;

    glutPostRedisplay();
}

// mouse movimento
void motion(int x, int y) {
    if (botaoPressionado) {
        anguloY += (x - ultimoX) * 0.5;
        anguloX += (y - ultimoY) * 0.5;

        ultimoX = x;
        ultimoY = y;

        glutPostRedisplay();
    }
}

void init() {
```

```
glEnable(GL_DEPTH_TEST);
glClearColor(0.0, 0.0, 0.0, 1.0);

glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluPerspective(60, 1.0, 1.0, 50.0);

glMatrixMode(GL_MODELVIEW);
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);

    glutInitWindowSize(800, 800);
    glutCreateWindow("Sala de Cinema 3D");

    init();

    glutDisplayFunc(display);
    glutMouseFunc(mouse);
    glutMotionFunc(motion);

    glutMainLoop();
    return 0;
}
```

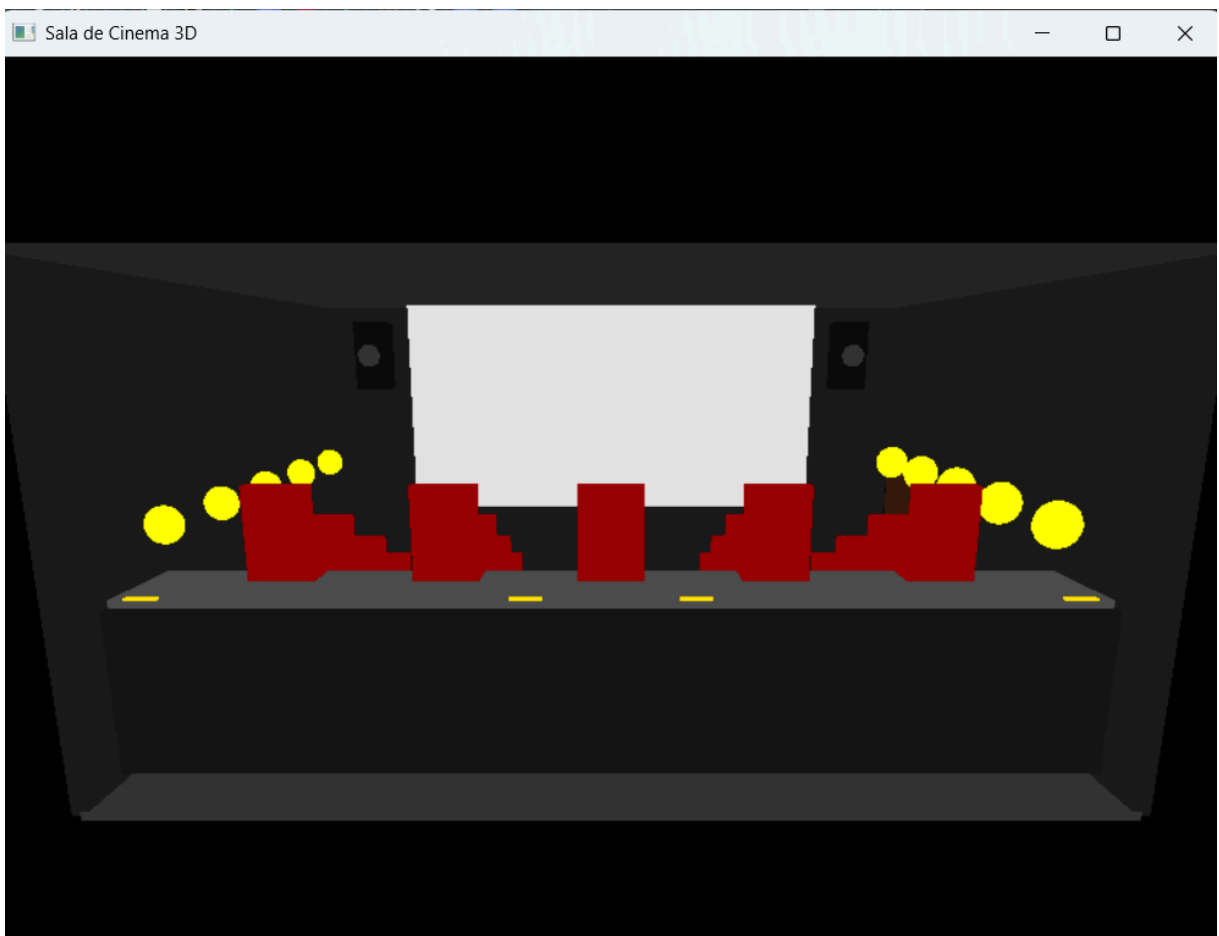


imagem base

